

Проектирую и реализую compiler/runtime и backend-системы, где корректность зависит от явных границ владения, состояния и жизненного цикла. В UniversalToolchain композиция сводится к одному immutable LanguagePlan до запуска; в VpnMediator состояния платежей и доступа отделены от reconciliation/recovery; в x86-64 backend решения register allocation проверяются независимым verifier и differential execution.

Architecture ownership

UniversalToolchain: один публичный LanguageCompiler разрешает композицию в immutable LanguagePlan; runtime материализует уже выбранный граф.

State & recovery

VpnMediator: явные payment/access states, идемпотентность, reconciliation, миграции, backup/restore, health gates и rollback.

Correctness boundaries

x86-64 backend: allocation проверяется независимым assignment verifier, а native execution сравнивается с эталонным interpreter oracle.

ОПЫТ

2026 — сейчас

ИСП РАН — инженер по статическому анализу

- Участвую в разработке SharpChecker — платформы статического анализа C#/NET в ИСП РАН.
- Работаю над задачами символьных вычислений в контексте статического анализа C#/NET.

июль — август 2026

МЦСТ — стажёр по разработке компиляторов

- Реализовал LICM-проход для LLVM 22 на C++; перенос выполняется только при выполнении условий по структуре цикла, обращениям к памяти, побочным эффектам и безопасности спекулятивного выполнения.
- Собрал Python-обвязку для сравнения поведения программы до/после оптимизации и проверки случаев с преобразованием и без него.

ПРОЕКТЫ

UniversalToolchain

Единственный публичный semantic planner разрешает зависимости и конфликты, выбирает providers, фиксирует порядок contributions и типизированные backend routes в одном immutable LanguagePlan до запуска runtime.

Архитектурные решения защищены pre-execution validation, exact manifest/ownership checks и parity interpreter/CIL; текущий exact manifest — 1 324/1 324.

VpnMediator

Явные payment/access states отделяют приём внешних событий от идемпотентного reconciliation и recovery; миграции и operational gates делают lifecycle-переходы проверяемыми.

Recovery включает backup/restore, health-gated переключение release и rollback; публичный кейс не раскрывает секреты и пользовательские данные.

x86-64 Compiler Backend & Register Allocation

Стратегия register allocation отделена от независимого assignment verifier до phi lowering и генерации SysV x86-64.

Семантика сгенерированного native-кода проверяется differential execution относительно эталонного интерпретатора; тесты покрывают ветвления, циклы, spills и parallel phi moves.

КОМПЕТЕНЦИИ

Архитектура

ownership/module boundaries, typed contracts, immutable planning, extensibility boundaries, explicit state machines, lifecycle/recovery, architecture verification

Compiler / Runtime platforms

.NET, LanguageCompiler/LanguagePlan, dependency/conflict resolution, provider selection, deterministic pass ordering, typed backend routes

State & Reliability

idempotency, reconciliation, migrations, backup/restore, health gates, rollback, failure containment

Проверка решений

regression tests, exact manifests, independent verifiers, interpreter/backend parity, differential execution

ОБРАЗОВАНИЕ

НИУ ВШЭ — Программная инженерия, 2026–2030

ДОСТИЖЕНИЯ

Доклад по архитектуре UniversalToolchain принят на LangDev'26; конференция пройдёт 8–9 октября 2026 в Малаге.

Москва / удалённо